LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING

CVE-2026-10879 DBI versions before 1.648 for Perl have a heap overflow when preparsing SQL statements with more than 9 binders

Vulnerability Report

Classification	TLP:CLEAR
Published	2026-06-12
Version	1.0
Author	Lost Boys Cyber
Report Type	Vulnerability Report

Published: 2026-06-12 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

CVE-2026-10879 DBI versions before 1.648 for Perl have a heap overflow when preparsing SQL statements with more than 9 binders

1. Executive Summary

CVE-2026-10879 is a heap-based buffer overflow in the Perl DBI module's `preparse()` logic. Public CNA, NVD, GitHub advisory, and oss-security reporting agree on the root cause: when SQL placeholders (`?`) are rewritten into numbered bind markers such as `:p10`, `:p100`, and higher, the vulnerable code allocates only three characters per binder even though placeholder counts above nine require additional bytes. In affected builds, SQL statements containing more than nine binders can therefore trigger an out-of-bounds write during statement preparsing.

The practical risk is highest where untrusted or indirectly attacker-influenced SQL statements can reach DBI-backed applications, middleware, job runners, or exposed service endpoints. Although the public CVSS 3.1 vector attached through NVD secondary scoring is Critical (9.8), the reviewed technical evidence more strongly supports a workflow-dependent application-layer exposure model than a blanket claim that every Perl service using DBI is trivially remotely exploitable. Defenders should treat this as a serious memory-safety flaw with potentially severe outcomes, while scoping exposure around applications that dynamically construct or relay SQL containing large numbers of bind parameters.

No reviewed source indicates inclusion in CISA's Known Exploited Vulnerabilities catalog or confirmed in-the-wild exploitation as of this report date. However, the issue is now well documented in public vulnerability feeds, release notes, and a directly linked patch diff. That combination lowers the barrier for proof-of-concept development and should move remediation and asset identification ahead of routine patch windows for internet-reachable or high-value DBI-dependent services.

Immediate defensive priorities are to identify DBI versions below 1.648, patch or rebuild to 1.648 or later, review applications that accept attacker-controlled query structure or large search/filter requests, and monitor for crashes or repeated failures in Perl services that invoke DBI statement preparation.

2. Intelligence Requirement

Question	Answer
What is the vulnerability?	A heap overflow in Perl DBI `preparse()` caused by under-allocating buffer space while expanding `?` placeholders into numbered binders such as `:p10`, `:p100`, and beyond.
Which versions are affected?	DBI versions earlier than 1.648.
What is the likely impact?	Memory corruption with possible process crash, denial of service, and potentially more severe code-execution outcomes depending on the embedding application and reachable input path.
What is the highest-priority exposure pattern?	Perl applications, APIs, batch systems, or middleware components that pass attacker-influenced SQL statements with more than nine bind variables into DBI.
What should defenders do first?	Inventory DBI-dependent assets, upgrade to 1.648+, review dynamic SQL generation paths, and monitor for DBI-related crashes or anomalous statement-preparation failures.

3. Source Review and Confidence

Source	Contribution	Confidence
CVE Program API record	Canonical affected-version range, CWE mapping, product metadata, and primary	High

	description	
NVD CVE 2.0 API	Normalized references, affected CPE, and secondary CVSS 3.1 score/vector	High
GitHub advisory GHSA-c7xw-cj86-m724	Human-readable severity context, repeated flaw description, linked patch reference	High
MetaCPAN DBI 1.648 release notes	Confirms the fix shipped in version 1.648 and names the issue as a preparse buffer overflow	High
Upstream patch `af79036c07aa9a45...`	Exact code-level remediation showing allocation changed from `strlen(statement) * 3` to `strlen(statement) * 6 + 16`	High
oss-security / Openwall disclosure	Public disclosure timing and concise problem statement from CPAN Security Group reporting	High
CISA weekly bulletin entry	Cross-ecosystem publication evidence and confirmation that public metadata was circulating broadly by 2026-06-09	Medium

Confidence is high that the flaw is real, affects DBI versions below 1.648, and was remediated upstream by increasing the allocation used during placeholder expansion. Confidence is only medium on broad remote-code-execution claims because the reviewed sources describe the memory-safety condition and severe CVSS scoring, but do not provide exploit details, a public proof of concept, or validated attack chains against common Perl web stacks. This report therefore treats the issue as high-priority and potentially high-impact, while avoiding unsupported certainty about exploit maturity.

4.1 Vulnerability Mechanics

The reviewed sources describe the same vulnerable pattern: DBI rewrites SQL placeholder characters into numbered binders of the form `:pN`. The old allocation logic assumed three characters per binder, which is only sufficient for single-digit placeholders such as `:p1`. Once placeholder counts exceed nine, the rewritten form grows to four or more characters and the destination buffer can be overrun during preparsing. The upstream patch directly addresses this by increasing the allocation from `strlen(statement) * 3` to `strlen(statement) * 6 + 16`.

4.2 Affected Scope

Dimension	Assessment	Evidence
Product	Perl DBI module	CVE Program API, NVD, GitHub advisory
Affected versions	Versions earlier than 1.648	CVE Program API, NVD CPE range
Fixed version	1.648	MetaCPAN release notes, patch-linked release
Weakness	CWE-787 Out-of-bounds Write	CVE Program API, NVD
Trigger condition	SQL statement preparsing with more than 9 binders	CVE description across CNA/NVD/GitHub/oss-security
Vulnerable file path	`DBI.xs` / `preparse()`	CVE Program API programFiles, upstream patch

4.3 Exposure Interpretation

Not every environment using DBI is equally exposed. The vulnerability becomes relevant when an application allows attacker-controlled or attacker-influenced query structure to reach DBI's placeholder-expansion path. Representative exposure patterns include:

- public-facing search or reporting endpoints that convert many user-supplied filters into bind-heavy SQL statements;
- middleware or ETL jobs that relay untrusted query templates into DBI-backed services;
- multi-tenant applications where one tenant can force the generation of unusually large placeholder lists through bulk-selection, import, or search APIs;
- internally exposed automation or admin tooling that accepts custom query or workflow definitions and later runs them through DBI.

4.4 Severity and Operational Assessment

Dimension	Assessment	Notes
Public severity	Critical (CVSS 3.1 9.8 via NVD secondary metrics)	`AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`
Technical confidence	High	Root cause and patch are public and specific
Exploit maturity	Low to Medium confidence	No reviewed public PoC or in-the-wild exploitation evidence
Most likely near-term outcome	Service crash / process instability	Common immediate effect for memory corruption during statement handling
Potential deeper impact	Plausible but unproven publicly	Memory corruption in a parser/preprocessor can enable stronger outcomes depending on process context
Remediation urgency	High	Fix is straightforward and public detail is sufficient for adversary study

5. Threat Context

The current public picture suggests a newly disclosed but not yet broadly weaponized supply-chain and application-dependency issue. The vulnerability sits in a common Perl database abstraction layer rather than in a single finished product, which means exposure depends on how downstream applications use DBI and what inputs they allow. This library-centric profile creates two common defensive blind spots: organizations may not realize where DBI is embedded, and teams may assume a base-OS package review is enough even when applications vendor or bundle Perl dependencies separately.

At present, the strongest attacker value appears in environments where DBI-backed services are internet reachable, process rich user-controlled filters, or run with permissions that make process compromise more consequential. Even if adversaries initially use the issue for denial of service rather than code execution, repeated crashes in business-critical query services, internal automation, or API layers can still create operational disruption and incident-response overhead.

5.1 Representative Attack Scenarios

Scenario	Path	Defensive Implication
Internet-facing application abuse	An attacker submits crafted requests that drive generation of SQL with more than nine bind placeholders, triggering the vulnerable `preparse()` path	Patch internet-reachable Perl apps first and monitor for crash/restart patterns
Authenticated tenant abuse	A low-privilege user leverages bulk filters,	Review multi-tenant business logic and

	exports, or saved-search features to induce oversized binder expansion	rate-limit query-heavy features
Internal workflow disruption	An internal job or admin tool accepts imported criteria/templates and later passes them into DBI	Include admin utilities and batch systems in asset inventories, not just front-end web apps
Embedded dependency lag	A product ships with DBI < 1.648 even after base package maintenance is completed elsewhere	Validate bundled Perl dependencies through SBOM or filesystem inspection

5.2 MITRE ATT&CK Mapping

ATT&CK mapping for a library vulnerability like this is necessarily scenario-dependent. The most defensible mappings are tied to how an adversary would operationalize the flaw rather than to the flaw itself.

Technique	Name	Rationale
T1190	Exploit Public-Facing Application	Appropriate when a remote application path allows crafted input to reach vulnerable DBI statement preparsing.
T1499	Endpoint Denial of Service	Reasonable if exploitation results in repeated crashes or process instability rather than confirmed code execution.
T1195	Supply Chain Compromise	Relevant for dependency-management and third-party product exposure tracking, though not as the direct exploitation technique.

6.1 Detection Coverage Summary

Signal	Telemetry Source	Why It Matters
Presence of DBI versions below 1.648	Package inventory, SBOM, container manifests, software asset databases	Best proactive method to identify exposure before crash telemetry appears
Perl service crashes or restart loops	EDR process telemetry, systemd/journald, application logs	Likely early symptom if the vulnerable path is exercised
Error strings around DBI statement preparation	Application logs, stderr capture, APM traces	Can expose failed exploit attempts or unstable query-building paths
Requests producing abnormally large filter/binder counts	Web logs, API telemetry, tracing, WAF logs	Helps identify reachable attack surfaces even before confirmed exploitation
Bundled Perl dependencies in application directories or containers	File inventory, build manifests, image scanning	Detects exposures missed by OS-centric patching workflows

6.2 Microsoft Defender KQL - Asset Inventory Hunt

```
// Identify Linux devices that appear to have Perl DBI installed below 1.648.
DeviceTvmSoftwareInventory
| where OSPlatform =~ "Linux"
| where SoftwareName has "DBI" or SoftwareName has "perl-DBI"
| extend ParsedVersion = tostring(SoftwareVersion)
| where ParsedVersion matches regex @"^([0-1]?\.[0-9]+.*)$"
| project DeviceName, SoftwareVendor, SoftwareName, SoftwareVersion, EndOfSupportStatus
| order by SoftwareVersion asc
```

6.3 Microsoft Defender KQL - Crash and Query-Stress Correlation

```
// Hunt for Perl/DBI-related crashes or unstable processes, then pivot to possible
// user-driven request pressure or error messages near the same time window.
let CrashTerms = dynamic(["segfault", "segmentation fault", "core dumped", "heap overflow", "buffer
overflow", "DBI", "preparse"]);
DeviceProcessEvents
| where Timestamp > ago(30d)
| where FileName in~ ("perl", "perl5", "plackup", "starman", "hypnotoad") or ProcessCommandLine has
"DBI"
| join kind=leftouter (
    DeviceEvents
    | where Timestamp > ago(30d)
    | where ActionType has_any ("Crash", "ApplicationCrash", "Fault") or AdditionalFields has_any
(CrashTerms)
    | project CrashTime=Timestamp, DeviceId, ActionType, AdditionalFields
) on DeviceId
| where isnotempty(CrashTime)
| project Timestamp, CrashTime, DeviceName, FileName, ProcessCommandLine, ActionType, AdditionalFields
| order by CrashTime desc
```

6.4 Splunk Hunt Query

```
(index=linux OR index=syslog OR index=app_logs)
("DBI" OR "preparse" OR "perl" OR "plack" OR "starman")
("segmentation fault" OR "core dumped" OR "heap overflow" OR "buffer overflow" OR "statement prepare"
OR "prepare failed")
| stats count min(_time) as first_seen max(_time) as last_seen values(host) as hosts values(process) as
processes values(source) as sources by message
| convert ctime(first_seen) ctime(last_seen)
| sort - count
```

6.5 Sigma Rule

```
title: Possible Perl DBI Preparse Crash Activity
id: 14fbf6f8-98a0-4d97-a264-cve-2026-10879
status: experimental
description: Detects Linux application or system logs that suggest Perl DBI statement-preparation
crashes or overflow-related failures.
logsource:
  product: linux
  category: application


detection:
  dbi_terms:
    message|contains:
      - 'DBI'
      - 'preparse'
      - 'prepare failed'
      - 'statement prepare'
  crash_terms:
    message|contains:
      - 'segmentation fault'
      - 'core dumped'
      - 'heap overflow'
      - 'buffer overflow'
      - 'memory corruption'
  condition: dbi_terms and crash_terms
falsepositives:
  - Benign application crashes unrelated to CVE-2026-10879
  - Test or QA environments intentionally exercising DBI error paths
level: medium
tags:
  - attack.t1190
  - attack.t1499
```

6.6 Threat-Hunting Leads

- Inventory internet-facing Perl applications and APIs that generate SQL dynamically, especially those supporting bulk filters, search builders, exports, or saved-query features.
- Review Perl application logs for prepare-time failures, sudden crash loops, or unexplained worker restarts after unusual query requests.
- Use SBOM or package inspection to determine whether third-party applications bundle DBI rather than inheriting a patched system package.
- Inspect source repositories for dynamic construction of long `IN (...)` lists or binder-heavy prepared statements that may be attacker-influenced.
- If suspicious activity is identified, preserve request samples, SQL templates, crash dumps, and exact DBI version evidence for follow-on validation.

7. Recommendations

Priority	Owner	Recommendation	Rationale
Immediate	Vulnerability Management / Platform	Upgrade Perl DBI to version 1.648 or later everywhere it is present.	Upstream fix is available and directly addresses the vulnerable allocation logic.
Immediate	Application Owners	Identify applications and services that allow user-driven query construction, bulk filters, or dynamic SQL generation through DBI.	Exposure depends on reachable statement-preparation workflows, not mere package presence.
High	Detection Engineering / SOC	Deploy package-inventory hunts plus crash and error-pattern monitoring for Perl/DBI services.	Best near-term way to identify both latent exposure and exploitation attempts.
High	Engineering	Add request-size, filter-count, and query-complexity guardrails where DBI-backed applications accept flexible user input.	Reduces exploit reachability even before all patching is complete.
High	Asset / Supply-Chain Management	Validate bundled Perl libraries in containers, appliances, and third-party apps rather than assuming OS-level remediation is sufficient.	Library vulnerabilities frequently persist in vendored application paths.
Medium	Incident Response	If suspicious crashes occur, collect application logs, core dumps, request traces, and software-bill-of-material evidence before service restart or redeploy.	Preserves evidence needed to distinguish benign instability from exploit attempts.

Additional analyst notes:

- Treat the public Critical CVSS score as a cue for patch urgency, but avoid overstating universal remote exploitability until a concrete attack path is validated in your environment.
- Prioritize internet-reachable Perl services and multi-tenant business workflows ahead of low-exposure internal utilities.

- Where immediate patching is not possible, temporarily constrain user-controlled query complexity and increase monitoring around DBI-backed endpoints.

8. References

- Microsoft Security Update Guide intake item, `CVE-2026-10879`
- https://msrc.microsoft.com/update-guide/vulnerability/CVE-2026-10879
- CVE Program API record, `CVE-2026-10879`
- https://cveawg.mitre.org/api/cve/CVE-2026-10879
- NVD CVE 2.0 API, `CVE-2026-10879`
- https://services.nvd.nist.gov/rest/json/cves/2.0?cveId=CVE-2026-10879
- GitHub Advisory Database, `GHSA-c7xw-cj86-m724`
- https://github.com/advisories/GHSA-c7xw-cj86-m724
- MetaCPAN DBI 1.648 release notes
- https://metacpan.org/release/HMBRAND/DBI-1.648/changes
- Upstream DBI patch `af79036c07aa9a457971c0f4136e37c85dc20978`
- https://github.com/perl5-dbi/dbi/commit/af79036c07aa9a457971c0f4136e37c85dc20978.patch
- Openwall / oss-security disclosure thread
- https://www.openwall.com/lists/oss-security/2026/06/06/4
- CISA Vulnerability Summary for the Week of June 1, 2026
- https://www.cisa.gov/news-events/bulletins/sb26-159